

Technical Design Document: Architecture and Implementation of a High-Fidelity Quranic Recitation Verification Platform

1. Executive Summary and System Philosophy

This Technical Design Document (TDD) articulates the comprehensive architectural blueprint, component specifications, and implementation strategy for a next-generation Quranic application. The proposed system is designed to provide real-time, automated verification of Quranic recitation (*Tajweed*) by synthesizing advanced mobile computing paradigms with scalable cloud-native infrastructure. The core objective is to bridge the gap between traditional oral transmission of the Quran and modern Artificial Intelligence (AI) by creating a system capable of detecting subtle phonetic nuances, elongation errors (*Madd*), and articulation attributes (*Sifaat*).

The architecture represents a fundamental paradigm shift from traditional hybrid mobile development. By leveraging the **React Native New Architecture**, specifically the **JavaScript Interface (JSI)** and **TurboModules**, the mobile client bypasses the asynchronous bridge bottleneck, enabling synchronous, high-frequency communication between the JavaScript UI layer and a native C++ audio processing core.¹ This allows for the integration of **Oboe** for ultra-low latency audio on Android and **CoreAudio** on iOS, sharing memory directly with on-device inference engines like **ONNX Runtime**.³

On the backend, the system employs a **Kubernetes-orchestrated microservices architecture** designed for high availability and elastic scalability. The backend facilitates complex acoustic modeling, heavy-duty verification tasks that exceed mobile capabilities, and a continuous learning pipeline managed by **Argo Workflows**. Data persistence utilizes a polyglot strategy, employing **PostgreSQL** for relational user data and **ClickHouse** for high-volume analytics of recitation telemetry.⁵ This document serves as the definitive reference for engineering teams, detailing the interaction between high-performance C++ modules, deep learning phoneme alignment algorithms, and distributed containerized infrastructure.

The guiding philosophy of this architecture is "**Latency is Quality.**" In a pedagogical feedback loop, any perceptible delay between the user's vocalization and the system's corrective feedback severs the cognitive association between action and error. Therefore, the design prioritizes "Zero-Copy" data transfer mechanisms, edge-first computing to minimize network round-trip dependence, and scalable intelligence that leverages the cloud only for

asynchronous model improvement or tasks exceeding device thermal envelopes.

2. Mobile Client Architecture: The Convergence of React Native and C++

The mobile client functions as the primary data acquisition point and user interface. The historical limitations of React Native—specifically the asynchronous "Bridge" that serializes messages to JSON—render the legacy architecture unsuitable for real-time, high-fidelity audio analysis. To achieve the requisite sub-10ms processing latency, this design mandates the adoption of the **React Native New Architecture** (Fabric and TurboModules), utilizing C++ as the unified logic layer across Android and iOS.

2.1 The JavaScript Interface (JSI) and Zero-Copy Architecture

The JavaScript Interface (JSI) is the linchpin of the new architecture. It facilitates a direct, synchronous communication channel between the JavaScript engine (Hermes) and the native C++ runtime. Unlike the legacy bridge, which required serializing complex data structures into JSON strings—a process that incurs significant CPU overhead and garbage collection pressure—JSI allows JavaScript to hold references to C++ "Host Objects" and invoke methods on them directly.¹

2.1.1 HostObjects and Memory Sharing

In the context of real-time audio processing, the system must handle streams of Pulse-Code Modulation (PCM) data at sampling rates of 16kHz or 44.1kHz. Passing these buffers from the native layer to the JavaScript layer for visualization (e.g., rendering a waveform or pitch graph) via the traditional bridge would result in dropped frames and UI stutter.

The proposed solution utilizes `jsi::ArrayBuffer` to implement a shared memory model. When the native C++ audio callback receives a buffer from the microphone hardware, it writes this data into a pre-allocated memory region (a ring buffer). This memory region is encapsulated within a C++ `HostObject`. Through JSI, the JavaScript runtime is granted a direct view of this memory as a `Float32Array` or `Uint8Array`. This mechanism is strictly "Zero-Copy," meaning the data is not duplicated in memory; rather, the pointer is shared.⁸ Benchmarks indicate that for large arrays and high-frequency calls, C++ direct calls via JSI maintain consistent performance, whereas the bridge architecture degrades exponentially as data volume increases.¹⁰

2.1.2 The Audio TurboModule

The Audio TurboModule acts as the central controller for the audio subsystem. Written in C++, it ensures portability and consistency across platforms.

- **Initialization:** The module is responsible for initializing the underlying native audio APIs (Oboe on Android, AVAudioEngine on iOS) and configuring the hardware for optimal sample rates and buffer sizes.³
- **JSI Bindings:** Upon startup, the module installs JSI bindings into the global JavaScript object. This allows the UI logic to execute commands such as `global.AudioEngine.start()` or `global.AudioEngine.readBuffer()` synchronously.¹¹
- **Concurrency Management:** To prevent blocking the main JavaScript thread—which handles UI rendering—the C++ module spawns a dedicated high-priority thread for the audio I/O callback and a separate worker thread for signal processing (VAD, Feature Extraction). Synchronization with the JS thread is managed via the `CallInvoker` or atomic flags within the shared ring buffer.¹²

2.2 High-Performance Audio Engine Strategy

The accuracy of the speech verification engine is inextricably linked to the quality of the input signal. The audio engine must provide a stable, low-latency stream, robust against the hardware fragmentation inherent in the mobile ecosystem.

2.2.1 Android: Oboe vs. Miniaudio

Android presents a unique challenge due to the diversity of hardware and audio drivers. While **Miniaudio** is a capable cross-platform library¹³, **Oboe** is the selected solution for this architecture due to its specific optimization for the Android environment.

- **API Selection:** Oboe acts as a wrapper that automatically selects the best available audio API. On Android 8.1+ (API 27+), it utilizes **AAudio**, a high-performance stream optimized for low latency. On older devices, it seamlessly falls back to **OpenSL ES**. This abstraction significantly reduces the maintenance burden.³
- **Latency Optimization:** Oboe enables "Exclusive Mode," which allows the application to bypass the internal Android mixer and communicate directly with the hardware abstraction layer (HAL). This is critical for achieving round-trip latency below 20ms, which is the threshold for perceptible delay.¹⁵
- **Quirks Management:** A decisive factor in choosing Oboe is its built-in `QuirksManager`. The Android ecosystem is plagued by device-specific bugs (e.g., incorrect sample rate reporting on certain Samsung devices or buffer size mismatches on Xiaomi). Oboe handles these edge cases transparently, ensuring stability across the fragmented device landscape.¹⁷

Feature	Oboe	Miniaudio	Analysis for Quranic App
Primary OS	Android	Cross-Platform	Oboe is purpose-built for

			Android's complexity.
API Backend	AAudio / OpenSL ES	AAudio / OpenSL / WASAPI / CoreAudio	Oboe's switching logic is more mature for Android.
Latency	Ultra-Low (Exclusive Mode)	Low	Oboe optimizes for the lowest possible Android latency.
Device Fixes	Extensive (QuirksManager)	Limited	Critical for preventing crashes on diverse user devices.
Integration	Native C++	C Header-only	Oboe fits naturally into the C++ TurboModule architecture.

2.2.2 iOS: Core Audio Integration

On iOS, the hardware landscape is homogeneous, and the software stack is consistent. The C++ Audio Engine interacts directly with the **Audio Unit** framework (specifically the RemoteIO unit) via a C++ wrapper. This wrapper exposes the same public interface as the Oboe implementation (e.g., `openStream`, `startStream`, `readFrame`), allowing the upper layers of the application to remain agnostic to the underlying OS.

2.3 On-Device Machine Learning: ONNX Runtime

To minimize server costs and uphold user privacy, the architecture dictates that Voice Activity Detection (VAD) and initial Phoneme Recognition be performed on the device. **ONNX Runtime (ORT)** is the inference engine of choice, offering a unified C++ API that abstracts the hardware acceleration capabilities of both Android and iOS.⁴

2.3.1 Model Quantization and Optimization

The acoustic model, likely a variant of **Wav2Vec2** or **HuBERT** fine-tuned for Arabic phonemes, typically requires significant memory and compute resources. A standard FP32 (32-bit floating point) model is too large for efficient mobile deployment.

- **Dynamic Quantization:** We employ dynamic quantization to convert the model weights from 32-bit floats to 8-bit integers (Int8). Research demonstrates that quantized Wav2Vec2 models can achieve a 4x reduction in size and a 2-3x speedup in inference latency with a Word Error Rate (WER) degradation of less than 1%.¹⁸
- **Execution Providers:** ORT is configured to use platform-specific execution providers:
 - **Android:** The **NNAPI** (Neural Networks API) is used to delegate computations to the device's NPU (Neural Processing Unit) or DSP.²⁰
 - **iOS:** The **CoreML** execution provider is utilized to leverage the Apple Neural Engine, ensuring high performance with minimal battery drain.²¹

2.3.2 Voice Activity Detection (VAD) Integration

Continuous ASR processing is battery-intensive. The system must intelligently discern between silence, background noise, and actual recitation. **Silero VAD** is integrated into the pipeline due to its high accuracy, sub-millisecond latency, and compact footprint (~2MB).²²

- **Signal Flow:** As audio buffers are read from the Oboe/CoreAudio stream, they are first passed to the Silero VAD model (running in ORT).
- **Gating:** Only audio frames with a speech probability exceeding a configurable threshold (e.g., 0.5) are buffered and forwarded to the heavy ASR model. This significantly reduces the thermal load on the device and bandwidth usage if cloud offloading is enabled.²³

3. The Quranic Verification Engine: Algorithmic Logic and Linguistics

The core differentiator of this application is its Verification Engine. Unlike standard Speech-to-Text (STT) systems that approximate the most probable sentence from a language model, Quranic verification requires the exact alignment of recited phonemes against a rigid, canonical text (the *Mushaf*), encompassing complex articulation rules known as *Tajweed*.

3.1 Acoustic Modeling: Wav2Vec2-XLSR Adaptation

The acoustic model is built upon **Wav2Vec2-Large-XLSR-53**, a massive multilingual model pre-trained on 53 languages, including Arabic.²⁵ This foundation allows the model to leverage cross-lingual phonetic representations, which is crucial for low-resource tasks.

3.1.1 Fine-Tuning and Diacritics

Standard Arabic ASR models are typically trained on Modern Standard Arabic (MSA) which is often undiacritized. However, Quranic meaning is heavily dependent on diacritics (*Tashkeel*). A change in a vowel (e.g., *Fatha* to *Kasra*) can invert the meaning of a verse.

- **Tokenizer Extension:** The model's tokenizer is modified to treat diacritized characters (e.g., 'ا', 'ب', 'بِ') as distinct tokens, or alternatively, to utilize a multi-head output where

one head predicts the character and another predicts the diacritic.²⁷

- **Dataset Strategy:** The model is fine-tuned on a composite dataset comprising the **Arabic Speech Corpus, Common Voice (Arabic)**, and domain-specific datasets like **EveryAyah** and the **Qur'anic Mispronunciation Benchmark (QuranMB)**.²⁹ This ensures the model learns the specific prosody and slow cadence of recitation.

3.2 Grapheme-to-Phoneme (G2P) and the Quranic Phonetic Script (QPS)

To verify pronunciation, the system cannot rely strictly on orthography. The Uthmani script of the Quran is semi-transparent; however, *Tajweed* rules introduce phonetic transformations that are not explicitly written (e.g., *Idgham* merges two letters, *Ikhfa* hides a sound).

- **Quranic Phonetic Script (QPS):** To address this, the architecture defines a specialized **Quranic Phonetic Script (QPS)**.³¹ This script maps the text to a sequence of phonemes that includes Tajweed attributes. For instance, the 'Lam' in "Allah" requires a heavy (*Tafkheem*) pronunciation if preceded by a Fatha, which QPS encodes explicitly (e.g., *l_heavy*), whereas a standard IPA transcription might just use /l/.
- **Rule-Based G2P:** A deterministic, rule-based G2P engine processes the requested Ayah text to generate the "Ground Truth Phoneme Sequence." This engine implements the rules of *Hafs an Asim* (the most common recitation style) to predict exactly how the verse should sound.³³

3.3 Alignment and Error Detection: The Tasmeea Algorithm

The verification logic employs a modified version of the **Tasmeea Algorithm**, which utilizes the **Needleman-Wunsch** algorithm for global sequence alignment.³¹ This approach is preferred over simple Levenshtein distance because it supports affine gap penalties and custom scoring matrices essential for phonetic analysis.

3.3.1 The Alignment Process

1. **Transcription:** The on-device ASR model processes the user's audio and outputs a "Hypothesis Sequence" of phonemes.
2. **Normalization:** Both the Hypothesis and the Ground Truth (from G2P) are normalized to a compatible phoneme set to ensure fair comparison.
3. **Matrix Calculation:** The Needleman-Wunsch algorithm computes an optimal alignment matrix.
 - **Custom Scoring:** The scoring matrix is weighted based on phonetic similarity. For example, substituting a 'T' (Ta) with a 'D' (Dal) is a common error due to shared articulation points (*Makhraj*). This error is penalized less severely than substituting 'T' with 'K', which are phonetically distinct. This nuance allows the app to distinguish between "close attempts" and "complete misses".³⁵
4. **Backtracking:** The algorithm backtracks through the matrix to generate the alignment

path, identifying specific Insertions, Deletions, and Substitutions.

3.3.2 Duration and Timing Analysis

Certain Tajweed rules, specifically *Madd* (elongation), are purely temporal. A standard ASR might transcribe the phonemes correctly but fail to detect that the user held the vowel for only 2 counts instead of the required 6.

- **CTC Timestamps:** The Wav2Vec2 model is trained using **CTC (Connectionist Temporal Classification)** loss. During inference, the model emits not just the phoneme but also the timestep of the emission.
- **Verification:** The system extracts the duration of vowels from these timestamps. If a phoneme aligns correctly with a "Madd" token in the Ground Truth, the system checks if the duration falls within the acceptable range (e.g., > 800ms for a 6-count Madd). If not, a "Timing Error" is flagged.³⁷

4. Backend Infrastructure: Scalability and DevOps

While the mobile client handles immediate feedback, the backend infrastructure is responsible for heavy lifting, data aggregation, user profile management, and the continuous improvement of the acoustic models. The backend is architected as a set of containerized microservices orchestrated by **Kubernetes**.

4.1 Cluster Architecture and Node Pools

The Kubernetes cluster is segmented into distinct node pools to optimize resource utilization and cost:

- **General Purpose Pool:** Hosts the API Gateway, Authentication Service, and Profile Service. These workloads are I/O bound and run efficiently on standard CPU instances.
- **Inference Pool:** Consists of GPU-accelerated nodes (e.g., NVIDIA T4 or A10G) dedicated to running the heavy server-side ASR and verification models. This pool utilizes the **Kubernetes Cluster Autoscaler** to scale down to zero when demand is low (e.g., outside of peak prayer times), significantly reducing operational costs.³⁸
- **Data Pool:** Dedicated nodes with high-I/O NVMe SSDs for database persistence (if self-hosted) and high-throughput logging.

4.2 Ingress and Traffic Management

Traefik is selected as the Ingress Controller for this architecture, preferred over NGINX for its cloud-native design.⁴⁰

- **Dynamic Configuration:** Traefik natively supports Kubernetes Custom Resource Definitions (CRDs), allowing for dynamic routing configuration without manual reloads. This is vital for a microservices environment where services scale up and down

frequently.

- **Protocol Support:** Traefik provides robust support for **gRPC** and **WebSockets**, which are essential for streaming audio data from the client to the cloud for secondary verification or data collection.
- **Observability:** Traefik's built-in metrics export to Prometheus allows for real-time monitoring of ASR request latencies and error rates, key KPIs for the application's performance.⁴²

4.3 MLOps Pipeline: Continuous Learning with Argo

A critical requirement for an AI-driven Quran app is the continuous improvement of the acoustic models. As the user base grows to include diverse accents (e.g., Southeast Asian, West African), the model must adapt. **Argo Workflows** is implemented as the orchestration engine for this machine learning pipeline.⁴³

- **Data Ingestion:** The pipeline is triggered efficiently by events (e.g., accumulation of 10,000 new verified logs).
- **Preprocessing:** Audio data is normalized, and silence is trimmed using the VAD logic.
- **Training:** Distributed training jobs are launched on the GPU node pool. This involves fine-tuning the base Wav2Vec2 model on the newly collected "in-the-wild" data.
- **Evaluation:** The candidate model is evaluated against a rigorous hold-out test set (QuranMB.v1). Metrics tracked include WER, CER (Character Error Rate), and PER (Phoneme Error Rate).³⁰
- **Canary Deployment:** If the new model demonstrates statistically significant improvement, it is packaged into a Docker container and gradually rolled out to a subset of users via Argo Rollouts.

4.4 Example Argo Workflow Definition

To illustrate the simplicity and power of Argo for this use case, the following YAML structure defines the retraining pipeline:

YAML

```
apiVersion: argoproj.io/v1alpha1
kind: Workflow
metadata:
  generateName: quran-asr-retrain-
spec:
  entrypoint: main-pipeline
  templates:
```



```

- name: main-pipeline
dag:
  tasks:
    - name: ingest-data
      template: data-loader
    - name: train-model
      dependencies: [ingest-data]
      template: pytorch-job
    - name: evaluate-model
      dependencies: [train-model]
      template: evaluator

- name: pytorch-job
container:
  image: pytorch/pytorch:1.13-cuda11.6
  command: ["python", "train_distributed.py"]
  resources:
    limits:
      nvidia.com/gpu: 2

```

This declarative approach ensures the training pipeline is reproducible and version-controlled.⁴⁵

5. Data Strategy and Analytics

The application generates two distinct categories of data: transactional state data (user profiles, progress) and high-volume event data (recitation logs, error telemetry). A "One Size Fits All" database strategy is insufficient.

5.1 Analytical Database: ClickHouse

For storing and analyzing recitation logs, **ClickHouse** is selected as the superior choice over TimescaleDB.⁵

- **Workload Characteristics:** The application generates a massive stream of append-only data: every phoneme spoken, every confidence score, every latency metric. For a user base of 1 million, this can result in billions of rows per month.
- **Columnar Efficiency:** ClickHouse's columnar storage format allows for extreme data compression and lightning-fast analytical queries. For example, a query such as "Identify the most common pronunciation error in *Surah Al-Fatiha* among users in Indonesia" requires scanning specific columns across billions of records. ClickHouse executes this type of OLAP (Online Analytical Processing) query orders of magnitude faster than row-based or hybrid stores like PostgreSQL/TimescaleDB.⁴⁷

- **Batch Ingestion:** ClickHouse is designed to handle high-throughput ingestion, capable of accepting millions of rows per second, preventing the database from becoming a bottleneck during peak usage times (e.g., Ramadan).

Feature	ClickHouse	TimescaleDB	Selection Rationale
Storage Model	Columnar	Row-based (Hypertable)	Columnar is vastly superior for aggregating specific metrics across billions of logs.
Ingestion Speed	Very High (Batch)	High	ClickHouse handles massive log streams more efficiently.
Compression	Excellent	Good	Critical for reducing storage costs of massive telemetry data.
SQL Support	Standard SQL	Full SQL (Postgres)	ClickHouse SQL is sufficient for analytics; full ACID is not required for logs.

5.2 Relational Database: PostgreSQL

For user management, **PostgreSQL** remains the industry standard. It handles:

- **User Profiles:** Account details, subscription status.
 - **Progress State:** Which Ayahs have been memorized, current streak.
 - **Social Graph:** Connections between students and teachers.
 - **ACID Compliance:** Essential for transactional integrity, especially regarding premium subscriptions or payments.
-

6. Security and Identity Management

6.1 Authentication Provider: Supabase Auth

For a modern B2C mobile application, **Supabase Auth** offers a streamlined, secure, and scalable solution, preferable to the complexity of maintaining a Keycloak instance.⁴⁸

- **Integration:** Supabase wraps the GoTrue API and provides a native React Native client that manages token persistence, refresh cycles, and social login flows (Google, Apple) out of the box.
- **Row Level Security (RLS):** A key architectural advantage is the integration with PostgreSQL's Row Level Security. We define security policies directly on the database tables (e.g., CREATE POLICY "User can see own data" ON profiles FOR SELECT USING (auth.uid() = user_id);). This ensures that even if the API layer is compromised or bypassed, the data remains secure at the storage level.

6.2 Privacy and Data Sovereignty

Given the religious and personal nature of the data, privacy is paramount.

- **Voice Data Handling:** The architecture enforces a strict policy where raw audio is processed in memory and discarded unless the user explicitly opts in to a "Quality Improvement Program." If stored, audio is encrypted at rest using AES-256.
- **Edge Processing:** The reliance on on-device inference is highlighted as a privacy feature. "Your voice, your device." Data leaving the device is minimized to anonymized embeddings or text logs, rather than raw voice samples.

7. Detailed Component Specifications

7.1 Module: AudioStreamer (C++ / JSI)

This module is the bridge between the hardware and the React Native application.

- **Responsibilities:**
 - Manage the lifecycle of the `oboe::AudioStream` (open, start, pause, close).
 - Implement the `oboe::AudioStreamCallback` interface to receive audio data.
 - Maintain a lock-free circular buffer (Ring Buffer) to decouple the high-priority audio callback thread from the lower-priority JS thread.
 - Expose a JSI method `read(size_t frames)` that wraps the buffer in a `jsi::ArrayBuffer`.
- **Code Structure Example (Conceptual):**

```
C++  
// AudioModule.cpp  
#include <jsi/jsi.h>  
#include <vector>
```

```

using namespace facebook::jsi;

// A class representing the shared buffer
class AudioBuffer : public HostObject {
public:
    std::vector<float> data; // The shared memory region

    AudioBuffer(size_t size) {
        data.resize(size);
    }

    // Expose the buffer to JS via a getter
    Value get(Runtime& rt, const PropNameID& name) override {
        if (name.utf8(rt) == "data") {
            // Create a TypedArray view over the C++ vector memory
            // This is the core "Zero-Copy" mechanism
            return ArrayBuffer(rt,
                               reinterpret_cast<uint8_t*>(data.data()),
                               data.size() * sizeof(float));
        }
        return Value::undefined();
    }
};

```

This snippet illustrates the mechanism: `nativeAudioBuffer.data` in JavaScript becomes a direct, typed view into the `std::vector` residing in the C++ heap.

7.2 Module: VerificationService (Python / Kubernetes)

This service runs in the cloud to handle deeper verification tasks or fallback inference.

- **Container Base:** pytorch/pytorch:1.13-cuda11.6-cudnn8-runtime.
- **Key Libraries:** torchaudio (for DSP), transformers (for Hugging Face models), fastapi (for the REST/WebSocket API), uvicorn (server).
- **API Contract:**
 - **Endpoint:** POST /verify/chunk or WS /verify/stream
 - **Input:** Binary audio chunk (OPUS encoded to save bandwidth), Metadata (Ayah ID, User ID).
 - **Output:** JSON payload containing:
 - `phonemes_detected`: The sequence recognized.
 - `alignment_score`: A confidence metric (0.0 - 1.0).
 - `errors`: A list of objects detailing type (Madd, Tajweed, Substitution), position (index in the verse), and severity.

8. Implementation Roadmap

Phase 1: Foundation (Weeks 1-8)

- **Mobile:** Establish the React Native New Architecture scaffolding. Implement the basic C++ TurboModule for Oboe (Android) and CoreAudio (iOS) playback and recording. Validate the JSI "Zero-Copy" shared memory pipeline with a simple visualizer.
- **Backend:** Deploy the Kubernetes cluster. Configure Traefik Ingress. Set up Supabase Auth and the PostgreSQL schema for user profiles.

Phase 2: The Core Engine (Weeks 9-16)

- **Mobile:** Integrate ONNX Runtime. Compile and quantize the Wav2Vec2 model for mobile (Int8). Integrate Silero VAD. Connect the audio ring buffer to the inference engine.
- **Algorithm:** Implement the G2P and Needleman-Wunsch alignment logic in Python (for backend validation) and port the lightweight version to C++ for the mobile client. Develop the Quranic Phonetic Script (QPS) mapping tables.

Phase 3: Cloud Verification & MLOps (Weeks 17-24)

- **Backend:** Deploy the heavy Inference Service on the GPU node pool. Implement WebSocket streaming for real-time cloud verification fallback.
- **MLOps:** Configure the ClickHouse cluster for log ingestion. Set up Argo Workflows for the retraining pipeline. Create the "Golden Set" dataset for automated model evaluation.

Phase 4: Optimization & Launch (Weeks 25+)

- **Optimization:** Profile memory usage of JSI buffers to prevent leaks. Tune VAD thresholds for different noise environments. Optimize the "Exclusive Mode" audio path on problematic Android devices.
- **Analytics:** Build Grafana dashboards on top of ClickHouse to visualize global error trends.
- **Beta:** Release to a limited cohort for "in-the-wild" acoustic data collection to refine the acoustic models before public launch.

9. Conclusion

This Technical Design Document outlines a robust, high-performance architecture that addresses the unique and demanding requirements of a Quranic verification application. By eschewing the traditional React Native Bridge in favor of **JSI and C++**, the architecture solves the latency bottleneck inherent in hybrid apps, enabling the seamless real-time feedback loop necessary for effective learning. By adopting **Oboe**, we resolve the historical fragmentation of

Android audio. By integrating **ONNX Runtime** and **Wav2Vec2**, we bring privacy-preserving, offline-capable AI directly to the user.

On the backend, the combination of **Kubernetes**, **Argo Workflows**, and **ClickHouse** establishes a self-reinforcing intelligence cycle. The system not only serves the user but learns from them, continuously refining its acoustic models to accommodate the vast diversity of global reciters. This architecture is not merely a collection of technologies; it is a cohesive system designed to aid the user in perfecting their recitation of the Holy Quran with the precision, respect, and accessibility it demands.

Works cited

1. About the New Architecture - React Native, accessed on January 3, 2026, <https://reactnative.dev/architecture/landing-page>
2. Understanding JSI in React Native | by Varun Kukade - Medium, accessed on January 3, 2026, <https://medium.com/@varunkukade999/understanding-jsi-in-react-native-9342c4f910fd>
3. Using Android low latency audio with React Native turbomodules | by Teemu Pohjanlehto, accessed on January 3, 2026, <https://blog.stackademic.com/using-android-low-latency-audio-with-react-native-turbomodules-e6828647928b>
4. Install ONNX Runtime | onnxruntime, accessed on January 3, 2026, <https://onnxruntime.ai/docs/install/>
5. ClickHouse® vs TimescaleDB: Best for real-time analytics 2025 - Tinybird, accessed on January 3, 2026, <https://www.tinybird.co/blog/clickhouse-vs-timescaledb>
6. Why I chose Postgres (TimescaleDB) over ClickHouse for storing 10M+ logs, accessed on January 3, 2026, <https://dev.to/pollilog/why-i-chose-postgres-timescaledb-over-clickhouse-for-storing-10m-logs-1e18>
7. What's the difference between bridging a module with C++ or with JSI in React Native?, accessed on January 3, 2026, <https://stackoverflow.com/questions/69501535/whats-the-difference-between-bridging-a-module-with-c-or-with-jsi-in-react-native>
8. From Files to Buffers: Building Real-Time Audio Pipelines in React Native - Callstack, accessed on January 3, 2026, <https://www.callstack.com/blog/from-files-to-buffers-building-real-time-audio-pipelines-in-react-native>
9. How to share an ArrayBuffer with c++ and js using JSI without copying? #1264 - GitHub, accessed on January 3, 2026, <https://github.com/facebook/hermes/discussions/1264>
10. c++ and react-native (2) : r/reactnative - Reddit, accessed on January 3, 2026, https://www.reddit.com/r/reactnative/comments/1f4jojf/c_and_reactnative_2/
11. Cracking Open the Magic of React Native + C++ for JSI Bindings - DEV

- Community, accessed on January 3, 2026,
https://dev.to/samuel_onireti_0e7ea18c79/cracking-open-the-magic-of-react-native-c-for-jsi-bindings-17i4
12. How to Add Custom JSI Bindings in Your TurboModule (and Why You Might Need Them), accessed on January 3, 2026,
<https://www.callstack.com/blog/how-to-add-custom-jsi-bindings-in-your-turbo-module-and-why-you-might-need-them>
 13. What open-source C++ audio libraries do you all recommend in 2024? I'm just looking for something that plays multiple audio files and music. No 3D sound needed. - Reddit, accessed on January 3, 2026,
https://www.reddit.com/r/gameenginedevs/comments/1bz6r9o/what_opensource_c_audio_libraries_do_you_all/
 14. Introducing Oboe: A C++ library for low latency audio - Android Developers Blog, accessed on January 3, 2026,
<https://android-developers.googleblog.com/2018/10/introducing-oboe-c-library-for-low.html>
 15. Audio latency for app developers | Android Open Source Project, accessed on January 3, 2026, <https://source.android.com/docs/core/audio/latency/app>
 16. Minimum latency - Android - JUCE Forum, accessed on January 3, 2026,
<https://forum.juce.com/t/minimum-latency/25971>
 17. is oboe future-proof? · google oboe · Discussion #1327 - GitHub, accessed on January 3, 2026, <https://github.com/google/oboe/discussions/1327>
 18. Effective and Efficient Mixed Precision Quantization of Speech Foundation Models - arXiv, accessed on January 3, 2026, <https://arxiv.org/html/2501.03643v2>
 19. Quantize Speech Recognition Models using NNCF PTQ API - OpenVINO™ documentation, accessed on January 3, 2026,
<https://docs.openvino.ai/2023.3/notebooks/107-speech-recognition-quantization-wav2vec2-with-output.html>
 20. (PDF) Wav2Vec2.0 on the Edge: Performance Evaluation - ResearchGate, accessed on January 3, 2026,
https://www.researchgate.net/publication/358603972_Wav2Vec20_on_the_Edge_Performance_Evaluation
 21. C++ | onnxruntime, accessed on January 3, 2026,
<https://onnxruntime.ai/docs/get-started/with-cpp.html>
 22. Silero VAD: pre-trained enterprise-grade Voice Activity Detector - GitHub, accessed on January 3, 2026, <https://github.com/snakers4/silero-vad>
 23. How to deploy to android? #38 - snakers4 silero-vad - GitHub, accessed on January 3, 2026, <https://github.com/snakers4/silero-vad/discussions/38>
 24. Live microphone inference · snakers4 silero-models · Discussion #2 - GitHub, accessed on January 3, 2026,
<https://github.com/snakers4/silero-models/discussions/2>
 25. (PDF) Improving Diacritical Arabic Speech Recognition: Transformer-Based Models with Transfer Learning and Hybrid Data Augmentation - ResearchGate, accessed on January 3, 2026,
https://www.researchgate.net/publication/389184832_Improving_Diacritical_Arabi

[c_Speech_Recognition_Transformer-Based_Models_with_Transfer_Learning_and_Hybrid_Data_Augmentation](#)

26. elgeish/wav2vec2-large-xlsr-53-arabic - Hugging Face, accessed on January 3, 2026, <https://huggingface.co/elgeish/wav2vec2-large-xlsr-53-arabic>
27. Improving Diacritical Arabic Speech Recognition: Transformer-Based Models with Transfer Learning and Hybrid Data Augmentation - MDPI, accessed on January 3, 2026, <https://www.mdpi.com/2078-2489/16/3/161>
28. Diacritic Recognition Performance in Arabic ASR - arXiv, accessed on January 3, 2026, <https://arxiv.org/pdf/2302.14022>
29. Wav2vec2 Large Xlsr 53 Arabic · Models - Dataloop, accessed on January 3, 2026, https://dataloop.ai/library/model/ionatasgrosman_wav2vec2-large-xlsr-53-arabic/
30. Towards a Unified Benchmark for Arabic Pronunciation Assessment: Qur'anic Recitation as Case Study - ISCA Archive, accessed on January 3, 2026, https://www.isca-archive.org/interspeech_2025/elkheir25b_interspeech.pdf
31. Tasmeea Algorithm for Quranic Transcript Verification - Emergent Mind, accessed on January 3, 2026, <https://www.emergentmind.com/topics/tasmeea-algorithm>
32. Hetchy/Quranic-Phonemizer: Tajweed-aware Quranic phonemizer / grapheme to phoneme (G2P) converter - GitHub, accessed on January 3, 2026, <https://github.com/Hetchy/Quranic-Phonemizer>
33. DialG2P: Dialectal Grapheme-to-Phoneme. Arabic as a Case Study - ACL Anthology, accessed on January 3, 2026, <https://aclanthology.org/2025.arabincnlp-main.38/>
34. A Grapheme to Phoneme Transcription System for Arabic Speech Synthesis - ResearchGate, accessed on January 3, 2026, https://www.researchgate.net/profile/Nadja_Bettayeb/publication/324784062_A_Grapheme_to_Phoneme_Transcription_System_for_Arabic_Speech_Synthesis/links/5ae248dbaca272fdaf8faf8e/A-Grapheme-to-Phoneme-Transcription-System-for-Arabic-Speech-Synthesis.pdf?origin=publication_detail
35. Sadeed: Advancing Arabic Diacritization Through Small Language Model - arXiv, accessed on January 3, 2026, <https://arxiv.org/html/2504.21635v2>
36. Needleman-Wunsch algorithm - Wikipedia, accessed on January 3, 2026, https://en.wikipedia.org/wiki/Needleman%E2%80%93Wunsch_algorithm
37. Rule-Based Embedded HMMs Phoneme Classification to Improve Qur'anic Recitation Recognition - MDPI, accessed on January 3, 2026, <https://www.mdpi.com/2079-9292/12/1/176>
38. AI/ML in Kubernetes Best Practices: The Essentials - Wiz, accessed on January 3, 2026, <https://www.wiz.io/academy/ai-security/ai-ml-kubernetes-best-practices>
39. Building a Kubernetes-Native AI Platform for the Enterprise - NexaStack, accessed on January 3, 2026, <https://www.nexastack.ai/blog/kubernetes-native-ai-platform>
40. Traefik vs. NGINX: Comparison and Practical Guide - Cast AI, accessed on January 3, 2026, <https://cast.ai/blog/traefik-vs-nginx/>
41. Why Traefik is the Best Ingress NGINX Replacement, accessed on January 3, 2026, <https://traefik.io/blog/migrate-from-ingress-nginx-to-traefik-now>
42. Nginx vs. Traefik vs. HAProxy: Comparing Kubernetes Ingress Controllers -

- vCluster, accessed on January 3, 2026,
<https://www.vcluster.com/blog/nginx-vs-traefik-vs-haproxy-comparing-kubernetes-ingress-controllers>
43. Argo Workflows, accessed on January 3, 2026,
<https://argoproj.github.io/workflows/>
44. Production MLOps with MLflow, Argo, and Kustomize - Jeffrey Taylor - Medium, accessed on January 3, 2026,
<https://jeftaylo.medium.com/mlflow-argo-workflows-and-kustomize-the-production-mlops-trinity-5bdb45d93f41>
45. Getting started with Training Models using Argo Workflows | Blog - Distrikt, accessed on January 3, 2026,
<https://distrikt.fi/blog/getting-started-with-training-models-using-argo-workflows>
46. Introduction to ArgoFlow. Orchestrating AI with Argo Workflows: | by Amar Shukla | Medium, accessed on January 3, 2026,
<https://medium.com/@amarshukla/introduction-to-argo-flow-886730ba2e4c>
47. ClickHouse vs TimescaleDB - InfluxData, accessed on January 3, 2026,
<https://www.influxdata.com/comparison/clickhouse-vs-timescaledb>
48. Supabase or Keycloak? A complete Guide - Skycloak, accessed on January 3, 2026, <https://skycloak.io/blog/supabase-or-keycloak-a-complete-guide/>
49. Supabase vs. Keycloak: Your SaaS' Best Auth - Cloudthrill, accessed on January 3, 2026, <https://cloudthrill.ca/supabase-vs-keycloak-saas>